Reporting No:2 Week No:2 From :21/05/2026 To :27/05/2026
College ID: 24IT016
Project Title: SproutSong Security Implementation

WEEKLY REPORT

Work done in last week (Attach supporting Documents):

1.

Part C includes roles and access control. We have to check various things regarding accessibility of different types of users on the site. We need to do these tests :-

Test	Expected Result
Student A tries to access Student B report URL	Access denied
Teacher from School A tries School B student API	Access denied
Parent changes childId in API request	Access denied
Expired token is used	Request rejected
Deleted user token is reused	Request rejected
User role changed but old token remains active rechecked	Token should be invalidated or
Admin-only API accessed by teacher	Access denied

Roles :-

Accountant, Admin, Analytics, Coordinator, Extra Curricular, Librarian, Management, Parents, Schools, Sports, Students, etc.

1) define the set of rules for each role in a document

2) Ask AI about how to prevent the access, set security measure from front-end or back-end or through middleware which redirects

3) Any changes to be done in the database or not. etc..

Generated the Access_Control_Security_Policy.docx.

Section 2 contains role definition, what each can access and what it can't.

Section 3 has role based access control.

Section 4 has methods to prevent unauthorised access which we still have to discuss in technical ways, how to implement.

Table in section 2 is important for us. It answers the set of rules for each roles.

Also check if there is any additional role which if we have missed out. It can also suggest and the implementation.

The few roles we can add:-

1. HR or Managing Staff

2. School Transport

3. Admission Person/ Receptionist

4. Exam Coordinator

5. Hostel Warden

6. IT admin/ System admin

I then generated some implementation related information.

I made the .js and other code files for the new roles. A summary of each file:-

shared/roles.js — The single source of truth. Copy it into both backend/config/roles.js and frontend/lib/roles.js. All roles, permissions, route guards, and login redirects live here. Change once, applies everywhere.

backend/models/User.js — Mongoose User model with role, schoolId, tokenVersion, and role-specific scoping fields (assignedHostelBlockIds, assignedExamIds, childIds, etc.)

backend/middleware/auth.js — 4 middlewares chained together: authenticate (JWT

verify + tokenVersion check) → authorize('module', 'action') → scopeToSchool (auto-appends school filter) → scopeToSelf (IDOR prevention)

backend/routes/newRoles.js — All Express routes for Hostel, Admissions, Marketing, Examinations with the correct middleware chain on each. Transport is stubbed with a comment.

backend/models/AuditLog.js — Logs every 403/401, auto-deletes after 90 days.

backend/scripts/seedRoles.js — Run once to create one test user per role.

frontend/hooks/useAuth.js — useAuth() hook giving you user, role, can('module', 'action'), is('role'), login, logout.

frontend/components/auth/withAuth.js — withAuth(Page, { allowedRoles }) HOC for page-level guards + <PermissionGate module="examinations" action="write"> for hiding UI elements.

2.

Tasks:-

Important API Rules:-

Every protected API should check:

req.user.id

req.user.role

req.user.schoolId

req.params.id

Never trust only frontend restrictions.

Bad practice:

```
// Only checking login
if (!req.user) return res.status(401).json({ message: "Unauthorized" });
```

Better practice:

```
// Check login + role + ownership
if (!req.user) return res.status(401).json({ message: "Unauthorized" });

if (!allowedRoles.includes(req.user.role)) {
 return res.status(403).json({ message: "Forbidden" });
}

if (record.schoolId.toString() !== req.user.schoolId.toString()) {
 return res.status(403).json({ message: "Access denied" });
}
```

Share the route file of core and auth back-end to claude and it will analyse and prepare the report.

Search for other code audit tools for our use case and prepare report.

I searched for the relevant code audit tools.

I first used OWASP ZAP. I did a vulnerability scan. Started for URL :

https://www.sproutsong.com/

We need the in depth analysis so please start the Code and API security as we discussed. Create a new dummy account in github and scan the repository to allow only required codes to be read by code audit tool.

1. Include any one role of front-end e.g. students, schools etc.
2. auth
3. backend-auth
4. core-backend
5. admin

include docker-compose and dockerfiles for each.

Found vulnerabilities using snyk.io (using dummy account). I added these 5 folders for now:

1. students
2. auth
3. backend-auth
4. core-backend
5. admin

API and Back-end security check includes:-
API routes
API security
Middleware
How to check the api params
Encrypt the response - how to make it possible?
Mongodb security checklist
Sensitive fields and field types - default values etc..

Search related to these and make a report.

Generated the Report.

3.

Tasks:-
Image shared contains security implementations we need to do. We need to know what it covers in Error handling , secure logging etc.

Claude can help in preparing the sample like:

API call from front-end
how to secure the payload and response
How to read the payload
format of the api response (return)

Make a document with examples and make sample codes.

Secure logging just means,

In the logs generated when any user fills the forms, the logs generated can be seen by the owners and admins who have access. These logs may sometimes have PERSONALLY IDENTIFIABLE INFORMATION like phone numbers, name, etc. This has been highlighted in the report we generated with the sheets for all 4 forms.
So basically, these fields shouldn't be stored as plaintext and should not be readable from the logs by anyone. We need to encrypt these names, phone numbers, etc. so no one can see it in readable form.

Generated report. Summary:-

Logging Strategy:
Three-layer architecture: Pino for all structured app/error logs, Morgan/pino-http for HTTP request logs, and MongoDB AuditLog for persistent security events (Medium) . Log level guide (ERROR → WARN → INFO → DEBUG → TRACE) with SproutSong-specific examples, and a standard log entry shape with all required fields.
Full logger.js with built-in redact paths (authorization header, cookie, password, token, OTP, email, phone, Aadhaar, card numbers). Pino's redact feature ensures sensitive fields are censored before any log is written — no credential leaks regardless of what controller code logs. (Google Cloud) Also includes pino-http middleware config and the AsyncLocalStorage request ID / correlation ID pattern.

And for masking the personally identifiable information:

We have to implement algorithms like:
HMAC-SHA256 (recommended for email, phone, Aadhaar) — deterministic and irreversible, so you can correlate logs without exposing the actual value. Best practice is to use keyed hashing to resist brute-force attacks.
Partial masking — phone last 4 digits, email j**@school.com
Full redaction — passwords, tokens, OTPs (zero log value)
AES-256-GCM — for audit log entries where PII must be stored for compliance but encrypted at rest, with negligible per-request overhead. (Nodejs-security)
Tokenisation — for payment data, replacing card numbers with opaque tokens via a vault. (Digiqt)
Format-Preserving Encryption (FPE) — for test/staging environments

We also have to add consent paragraphs with checkbox for user to agree. And have to store that as proof, alongwith timestamp of agreement by user.

Generated sample codes...
Here's what's in the zip — 10 files across a complete implementation:
backend/config/
logger.js — Single Pino logger instance with all 20+ REDACT_PATHS pre-configured (passwords, tokens, OTPs, email, phone, Aadhaar, card numbers, wildcards)
logRotation.js — pino-roll file rotation fallback (daily + 50MB threshold, 30-day retention)
backend/middleware/
httpLogger.js — pino-http middleware; logs route patterns not actual param values, strips query strings and headers
requestId.js — AsyncLocalStorage-based correlation ID; getLogger() returns a child logger with requestId + userId + schoolId + role auto-included
backend/utils/
piiMask.js — hmacMask(), partialMask(), maskEmail(), maskIp(), maskAadhaar(),

sanitiseKeys() — all algorithms from Section 5 of the guide
auditEncrypt.js — AES-256-GCM encryptForAudit() / decryptFromAudit() with versioned envelope (v1:<iv>:<tag>:<cipher>) for key rotation support
backend/models/
AuditLog.js — Mongoose schema with 90-day TTL index, encryptedTarget field, compound indexes, and a AuditLog.record() static method
backend/controllers/
authController.js — Login/logout/password-change with annotated ✅ correct vs ❌ bad patterns
studentController.js — Student fetch, enrolment, and counselling record access with mandatory audit entries
tests/secureLogging.test.js — 40+ assertions covering all masking functions, AES round-trips, and compliance checks (known PII strings verified to never appear in log output).

Attached MongoDB Security Checks image. Generate report based on it.

Report Generated.

4.
Created MongoDB Security boilerplate zip, which contains sample codes for MongoDB Security Implementation. These will serve helpful to the development team.

Summary of the zip file:-

config/
db.js — Mongoose connection with strict: true, strictQuery: true, debug disabled in prod, TLS support, safe error logging (never logs the URI)
mongod.conf — Hardened server config with bindIp: 127.0.0.1, authorization: enabled, quiet mode, TLS block ready to uncomment
middleware/
mongoSanitize.js — Strips $ and . operators from all request inputs before they hit your controllers, with logging of injection attempts
utils/
fieldEncryption.js — AES-256-GCM encrypt/decrypt with versioned v1:iv:authTag:data envelopes, plus an encryptedField() Mongoose schema helper that auto-encrypts on save and decrypts on read
sensitiveFieldHelpers.js — bcrypt password hashing, OTP generation/hashing/verification, reset token lifecycle, and sanitizeAadhaar() that stores only the last 4 digits
models/
User.js — Full User model with every sensitive field pattern: password (select: false + bcrypt), tokenVersion, OTP fields, reset token fields, Aadhaar last 4 only, toJSON that strips everything sensitive
AuditLog.js — TTL-indexed collection (90 days), static helpers logDenial() and logSensitiveAccess() with AES-encrypted target
SchemaTemplate.js — Copy-paste base for every new model
plugins/auditPlugin.js — Attach to any schema to auto-log writes and deletes
scripts/
generateKeys.js — Generates FIELD_ENCRYPTION_KEY and HMAC_SECRET_KEY
setupMongoUsers.js — Creates app / admin / backup users with correct least-privilege roles

verifySetup.js — Automated pre-deployment checker (env vars, user permissions, encryption, sensitive field helpers)
backup.sh — Daily GPG-encrypted mongodump → S3 cron script
tests/
mongodbSecurity.test.js — 40+ tests covering encryption round-trips, tamper detection, bcrypt, OTP lifecycle, reset tokens, Aadhaar sanitization, and security invariants

---

I then created the File Upload Security Guide Document.

Part A — Overview & Scope — context on SproutSong's platform, all 7 document upload types (marksheets, student photos, school docs, certificates, counselling reports, internship reports, teacher documents), and why file upload security is critical under DPDP.
Part B — Security Controls Table — the full controls table from your screenshots (allowed extensions, MIME validation, file size, virus scan, private bucket, signed URL, UUID naming, no executables, access control) with Required vs Recommended status and which contexts each applies to.
Part C — Allowed File Types Per Context — exact allowed extensions, MIME types, and max file sizes for each upload context in SproutSong.
Part D — Storage Architecture — the correct vs incorrect path structure (your File Access Rule with the Bad/Better examples), private bucket configuration requirements, AES-256 encryption, CORS policy, access logging, and versioning.
Part E — File Access Control — the full access matrix (counsellor, school admin, super admin, student/parent, unauthenticated), the 6-step server-side verification flow for every file download, and the IDOR prevention explanation.
Parts F–G — Implementation Code — ready-to-use Node.js/Express code for multer + MIME validation, S3 private bucket upload with UUID naming, the secure IDOR-protected file access route, and the MongoDB schema rules.
Part H — Virus Scanning — ClamAV implementation and AWS Lambda-based cloud-native scanning workflow.
Part I — Cascade Deletion — DPDP-compliant erasure code that deletes both the S3 object and MongoDB record, with the 72-hour SLA requirement.
Part J — Pre-Deployment Checklist — 20-item verification checklist the team must sign off before any file upload feature goes live.

5.
I created a sample code boilerplate zip for File Upload Security Implementation, which would serve as samples for the development team.

NOTE:- We also need to avoid files with 2 extensions.

Next task is about logging the success and failure and some critical issues which might occur or occurred...

I then created the password, OTP, session security guide.

Summary:-

Section 1 — Password Policy — Rules table (min 10 chars, complexity, bcrypt rounds ≥ 12, reset expiry, failed login limit,

MFA per role), plus a full Joi password schema with HaveIBeenPwned k-anonymity check implementation.
Section 2 — OTP Security — Rules table covering length, generation (crypto.randomInt vs Math.random), storage, expiry, used flag, rate limiting, and attempt limits. Full OTP lifecycle code: generate → hash → store payload → verify with attempt counter.
Section 3 — JWT Security — The full controls table from your screenshots (expiry, refresh token, blacklist, HttpOnly, Secure flag, SameSite, payload rules, secret rotation). Code for signing tokens with safe payload (no email/name/phone), and setting all cookies with correct flags.
Section 4 — Refresh Tokens — MongoDB RefreshToken model storing SHA-256 hash (not raw token), with TTL index, used flag, family field for rotation attack detection, and automatic replay detection that revokes the entire token family.
Section 5 — Token Blacklist — Two-mechanism approach: tokenVersion for bulk invalidation on logout/role change/password reset, plus jti blacklist in Redis for single-device logout. Covers the fallback if Redis is unavailable.
Section 6 — Failed Login & Lockout — Full login controller with 5-attempt lockout, 15-minute cooldown, and the same generic error message for wrong email AND wrong password to prevent user enumeration.
Section 7 — MFA — TOTP setup with otplib (secret encrypted with AES-256-GCM at rest, QR code generation), plus a role-based MFA matrix (TOTP for admins, SMS OTP for new device logins for parents/students).
Section 8 — Bad vs Good — 8-row comparison table covering all the key antipatterns.
Section 9 — Master Checklist — 35 items across Password, OTP, JWT, Token Invalidation, Login Security, and MFA sections.

This marked the conclusion of Part G as per the master document. So, I created a summarizing report deocument covering the details about all tasks performed from part A to G.

6.
I've made a guide like document that can be shared with development team. This has section wise security implementation details.

Then I made a sample code boilerplate zip for session security implementation. These will be helpful to the development team. This refers to the Part G document from yesterday.

I made another sample code boilerplate zip for Logging & Monitoring Implementation. These will be helpful to the development team. This refers to the Part H.

Reason for incomplete work:

1. No Incomplete Work

Plans for next week:

1. Breach Prevention, AI prediction safety, and moving ahead as per master document.

References:

1. GitHub Repository (Check branch named Week-2):-

https://github.com/Rudra733/Internship-Daily-Reports/tree/Week-2

Signature of External Guide **Signature of Internal Guide**

Student Id: 24IT016
Student Name: Dodiya Rudra Ketansinh